



UnB

Universidade de Brasília
Departamento de Ciências da Computação
CIC0202 - Programação Concorrente, Turma 01

Matheus Duarte da Silva – 211062277

Sincronização entre Processos Trabalho 1

Prof. Dr. Eduardo Adílio Pelinson Alchieri

Brasília
2026

1 Introdução

Historicamente, pela Lei de Moore (MOORE, 1965), o ganho de desempenho computacional deveria crescer a cada dois anos conforme o tamanho dos transistores iriam diminuindo, acabou esbarrando em limites físicos do *hardware*, tornando a otimização de *software* essencial para contornar esses gargalos (CONNER-SIMONS, 2020).

A adoção de um paradigma de programação concorrente (SUTTER et al., 2005) garantiu um alto *throughput* nos sistemas modernos, mas transferiu o gargalo de desempenho da CPU para as operações de Entrada e Saída (I/O). Em infraestruturas corporativas, isso se reflete diretamente no limite de conexões simultâneas suportadas por bancos de dados (GUPTA, 2025).

Operações simultâneas e pesadas, como rotinas de Extração, Transformação e Carregamento (ETL) e consultas de *Business Intelligence* (BI), impossibilitam a manutenção de inúmeras conexões abertas. A solução arquitetural padrão é o *Database Connection Pool* (DCP), que restringe as conexões ativas. Segundo Sobri et al. (2022), *pools* bem dimensionados podem reduzir o tempo de resposta de 312 ms para 187 ms e o consumo de CPU de 78% para 42% sob alta demanda.

Diante deste contexto, este trabalho objetiva modelar a resolução de um problema de contenção em um *pool* de conexões limitado.

2 Formalização do Problema Proposto

A partir do cenário mostrado, o problema a ser solucionado consiste no desenvolvimento de uma solução para tratar problemas de comunicação entre processos através de uma memória compartilhada, focando em um mecanismo de controle de concorrência para um *pool* de conexões de banco de dados. O sistema é acessado concorrentemente por dois grupos de *threads* com comportamentos antagônicos, gerando um cenário propício a condições de corrida (*race conditions*):

- **Escritores:** Processos responsáveis pela alteração do estado do banco de dados, representados por rotinas de testes e *pipelines* de ETL.
- **Leitores:** Processos que apenas consultam o estado do banco, representados por serviços de *Business Intelligence* (BI) que executam consultas pesadas e extração de métricas.

2.1 Restrições e Regras de Negócio

Para garantir a consistência dos dados e a estabilidade do servidor sob alta demanda, é preciso utilizar na resolução do problema utilizando mecanismos de sincronização, respeitando essas três regras da arquitetura:

1. **Limite de Capacidade (Gargalo de I/O):** O banco de dados suporta um número máximo de N conexões simultâneas.
2. **Exclusão Mútua Direcional:** Para evitar leituras que estejam desatualizadas ou incompleta, e garantir o isolamento das transações. O banco opera em um sistema de **fluxo unidirecional alternado**.
3. **Política de Escalonamento entre Processos:** O sistema implementa um escalonamento baseado em cotas para evitar o monopólio do banco de dados por um único grupo.

2.2 Casos Críticos

Há alguns casos específicos que esse problema possui e será tentado mitigar esses casos, sendo eles:

- ***Race Conditions*:** Sem o bloqueio adequado durante a alteração das variáveis de estado do *pool*, múltiplas *threads* poderiam estourar o limite de conexões.
- ***Starvation*:** Em um modelo tradicional de Leitores e Escritores, um fluxo massivo de consultas de BI poderia impedir indefinidamente as rotinas de ETL de atualizarem o banco.
- ***Deadlock*:** Risco caso as lógicas de bloqueio e sinalização da inversão de fluxo falhem, resultando em processos aguardando eternamente pela liberação da oportunidade sem a devida comunicação (*broadcast*).

3 Algoritmo Desenvolvido para Solução do Problema

3.1 Mecanismos de Sincronização

A implementação será realizada em linguagem **C**, usando a biblioteca **POSIX Pthreads**. Toda a parte de manipulação e tratamento dos fluxos será feito com essas funcionalidades da biblioteca:

- ***Locks (Mutexes)*:** Garantirão a exclusão mútua na verificação e atualização das variáveis globais, como a direção atual do fluxo (Leitura/Escrita) e os contadores.
- **Variáveis de Condição:** Atuarão como semáforos de sinalização direcional. Se a cota K for atingida e houver processos adversários aguardando, as *threads* do fluxo atual entram em estado de bloqueio passivo (*sleep*) e emitem um sinal para acordar o grupo oposto.

- **Semáforos (*Semaphores*):** Um semáforo de contagem modelará o limite de capacidade física (N) do *pool*, restringindo a quantidade de *threads* do grupo ativo que podem realizar operações no banco de dados simultaneamente.

3.2 Estrutura de Estado do Pool

A base do mecanismo de sincronização reside na estrutura `connection_pool_t`, responsável por manter o estado global e centralizar controles de concorrência. Conforme detalhado no Trecho de Código 1, a estrutura armazena as primitivas da biblioteca Pthreads e os contadores de estado.

Trecho de Código 1: Estrutura de dados do Pool de Conexões

```
1 typedef struct {
2     pthread_mutex_t lock;
3     pthread_cond_t cond_readers;
4     pthread_cond_t cond_writers;
5     sem_t sem_connections;
6
7     uint8_t active_readers;
8     uint8_t active_writers;
9     uint8_t waiting_readers;
10    uint8_t waiting_writers;
11
12    uint8_t consecutive_readers;
13    uint8_t consecutive_writers;
14
15    uint32_t total_connections;
16 } connection_pool_t;
```

O campo `lock` garante a exclusão mútua necessária para qualquer leitura ou alteração nos contadores. As variáveis de condição (`cond_readers` e `cond_writers`) atuam como filas de espera para as *threads* bloqueadas. O limite físico de conexões com o banco de dados (N) é gerido pelo semáforo `sem_connections`, que atua como uma maneira a mais de bloqueio direcional. Os contadores gerenciam a quantidade de *threads* em diferentes estados (ativas ou em espera) e monitoram o fluxo contínuo por meio das variáveis `consecutive`, que são essenciais para o escalonamento.

3.3 Lógica de Controle de Acesso e Escalonamento

O controle de entrada no *pool* implementa tanto a exclusão mútua direcional quanto a prevenção contra *starvation*. O Trecho de Código 2 mostra esse comportamento sob a perspectiva das *threads* de Leitura (BI), a lógica para as *threads* de Escrita (ETL) são parecidas.

Trecho de Código 2: Lógica de bloqueio e escalonamento para Leitores

```
1 bool quota_exhausted = (pool->consecutive_readers >= MAX_POOL_REQUESTS
    && pool->waiting_writers > 0);
2
3 while (pool->active_writers > 0 || quota_exhausted) {
4     pool->waiting_readers++;
5     pthread_cond_wait(&pool->cond_readers, &pool->lock);
6     pool->waiting_readers--;
7
8     quota_exhausted = (pool->consecutive_readers >= MAX_POOL_REQUESTS &&
        pool->waiting_writers > 0);
9 }
10
11 pool->active_readers++;
12 pool->consecutive_readers++;
13 pool->consecutive_writers = 0; // Reseta o contador do grupo oposto
14 pthread_mutex_unlock(&pool->lock);
15
16 sem_wait(&pool->sem_connections);
```

Ao tentar acessar o banco, a *thread* adquire o **lock** e avalia a condição de entrada por meio de um laço **while**. O acesso é negado se houver Escritores ativos, que garante a exclusão mútua direcional, ou se a quantidade máxima de leituras consecutivas (K) tiver sido atingida e existirem Escritores aguardando na fila (**quota_exhausted**). Esse segundo critério é o núcleo da mitigação de *starvation*, pois ele obriga o fluxo atual a ser interrompido, invertendo a prioridade de atendimento do *pool*.

Caso a *thread* seja bloqueada, ela incrementa o contador de espera e dorme, com o **pthread_cond_wait**, até que seja sua vez novamente de executar sua devida ação. Ao ser acordada e validar positivamente as condições, a *thread* contabiliza seu acesso, zera a contagem consecutiva do grupo oposto e libera o **lock**. Imediatamente após, ela tenta conseguir uma conexão com o pool através da chamada **sem_wait**. Essa separação entre a permissão lógica e a aquisição do recurso físico otimiza a concorrência.

3.4 Liberação de Recursos

A fase de saída encerra o ciclo de vida das transações. Diferente da entrada, o processo de devolução da conexão deve, obrigatoriamente, sinalizar a disponibilidade para eventuais *threads* adormecidas, evitando assim a ocorrência de *deadlocks*.

Trecho de Código 3: Liberação de recurso e emissão de sinal de disponibilidade

```
1 void exit_read_pool(connection_pool_t *pool, const uint8_t thread_id) {
2     sem_post(&pool->sem_connections); // Devolve a conexão física
3
4     pthread_mutex_lock(&pool->lock);
5     pool->active_readers--;
6
7     if (pool->active_readers == 0) {
8         pthread_cond_broadcast(&pool->cond_writers);
9     }
10
11     pthread_mutex_unlock(&pool->lock);
12 }
```

Como demonstrado no Trecho de Código 3, o primeiro passo da liberação é a devolução do espaço físico no banco utilizando `sem_post`. Em seguida, em posse do `lock` da estrutura, o número de leitores ativos é decrementado. Uma validação importante acontece neste momento: se o contador de atividades do fluxo atual chegar a zero (`active_readers == 0`), significa que a ponte está completamente vazia. Quando isso ocorre, é acionado o comando `pthread_cond_broadcast`, emitindo um sinal que acorda todas as *threads* do fluxo oposto que estavam bloqueadas.

4 Conclusão

O trabalho conseguiu cumprir seu objetivo ao solucionar o gerenciamento de um *pool* de conexões sob a demanda de rotinas de ETL e BI. A modelagem abstraiu com sucesso os desafios práticos de concorrência, como condições de corrida, *starvation* e gargalos de I/O.

A implementação em C, utilizando a biblioteca POSIX Pthreads, orquestrou o acesso compartilhado de maneira segura. Semáforos delimitaram a capacidade física do banco, enquanto a combinação de *mutexes* e variáveis de condição garantiu o isolamento transacional e a exclusão mútua direcional. Além disso, a política de escalonamento baseada em cotas consecutivas provou-se o grande diferencial da solução, mitigando ativamente o risco de *starvation* ao impedir o monopólio do *pool* por um único tipo de requisição.

Por fim, conclui-se que a aplicação rigorosa de mecanismos de sincronização é vital tanto para a integridade dos dados quanto para a otimização de arquiteturas de *software* modernas. O algoritmo demonstrou robustez na coordenação entre processos, validando a eficácia técnica do desenvolvimento de *middlewares* de concorrência em sistemas corporativos críticos.

Referências

CONNER-SIMONS, A. *If transistors can't get smaller, then coders have to get smarter* — *Massachusetts Institute of Technology*. 2020. Disponível em: [⟨https://news.mit.edu/2020/mit-csail-computing-technology-after-moores-law-0605⟩](https://news.mit.edu/2020/mit-csail-computing-technology-after-moores-law-0605).

GUPTA, R. Optimizing database performance through efficient connection management. *World Journal of Advanced Research and Reviews*, GSC Online Press, v. 26, 2025. Disponível em: [⟨http://dx.doi.org/10.30574/wjarr.2025.26.1.1111⟩](http://dx.doi.org/10.30574/wjarr.2025.26.1.1111).

MOORE, G. *Moore's law*. 1965. Disponível em: [⟨https://www.intel.com/content/www/us/en/history/virtual-vault/articles/moores-law.html⟩](https://www.intel.com/content/www/us/en/history/virtual-vault/articles/moores-law.html).

SOBRI, N. et al. A study of database connection pool in microservice architecture. *JOIV : International Journal on Informatics Visualization*, v. 6, p. 566, 08 2022.

SUTTER, H. et al. The free lunch is over: A fundamental turn toward concurrency in software. *Dr. Dobbs's journal*, v. 30, n. 3, 2005.

A Código Fonte

O código-fonte deste projeto está disponível no GitHub através do link:

[⟨https://github.com/smmstakes/pc-trabalho-1⟩](https://github.com/smmstakes/pc-trabalho-1)